MODULE 14 · DOMAIN 2 - CONTAINER ORCHESTRATION

Services and DNS

Every Service type, when to use each, and how CoreDNS names them.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 14 of 35

admin@sthithapragna.com

A Service is **a stable name and address** in front of a changing set of Pods.

A Service for the booking API

```
1 apiVersion: v1
2 kind: Service
3 metadata: {name: booking-api, namespace: quayside}
4 spec:
5   selector: {app: booking-api}
6   ports:
7   - name: http
8     port: 80
9     targetPort: 8080
```

5 Which Pods are behind it: an equality selector.

8 port: what clients connect to on the Service.

9 targetPort: where the container actually listens.

ON THE EXAM

No type given, so this is a ClusterIP Service: reachable only inside the cluster.

Service types at a glance

TYPE	REACHABLE FROM	HOW
`ClusterIP (default)`	Inside the cluster	A stable virtual IP
`NodePort`	Outside, via any node	A port from 30000-32767 opened on every node
`LoadBalancer`	Outside, via one address	A cloud load balancer in front of the NodePorts
`ExternalName`	Inside, pointing out	A DNS alias to an outside hostname; no proxying
`Headless (clusterIP: None)`	Inside the cluster	DNS returns the Pods' own IPs; no virtual IP

ON THE EXAM

Four types, plus headless as a variation of ClusterIP. Most questions ask which one fits.

Each type builds on the one before

01
Client
On the internet.

›

02
LoadBalancer
The cloud's load balancer, one external address.

›

03
NodePort
Traffic arrives at a port on some node.

›

04
ClusterIP
kube-proxy's rules pick a backend.

›

05
Pod
A ready Pod, possibly on another node.

ON THE EXAM

A LoadBalancer Service also has a NodePort and a ClusterIP. Each layer adds reach on top of the last.

ClusterIP versus NodePort

CLUSTERIP

Internal only

- The default
- A virtual IP inside the cluster
- For service-to-service calls
- Not reachable from outside

NODEPORT

A door on every node

- Same port open on every node
- Reach it at any node IP and port
- High port numbers only
- Clients must know node IPs

THE TIEBREAKER

Internal traffic: ClusterIP. Quick outside access without a cloud load balancer: NodePort.

NodePort versus LoadBalancer

NODEPORT

You bring the front door

- No external load balancer
- Clients track node addresses
- Works anywhere, including bare metal
- Fine for testing, awkward for users

LOADBALANCER

The cloud builds it

- One stable external address
- Created by the cloud-controller-manager
- Needs cloud integration, or MetalLB
- One load balancer per Service: cost

THE TIEBREAKER

Production internet traffic on a cloud: LoadBalancer, or an Ingress or Gateway in front of many Services.

externalTrafficPolicy: Cluster versus Local

CLUSTER (DEFAULT)

Any node forwards

- Every node accepts the traffic
- May hop to a Pod on another node
- Client IP replaced by a node IP
- Even spread across Pods

LOCAL

Only nodes with local Pods

- Only nodes running a Pod accept it
- No extra hop
- The real client IP is preserved
- Spread can be uneven

THE TIEBREAKER

The app needs to see the real client IP: externalTrafficPolicy: Local.

ExternalName, and Services without selectors

- **ExternalName:** the Service's DNS name becomes an alias (CNAME) for an outside hostname.
- **No selector, no proxying:** pure DNS. Apps use an in-cluster name for something outside.
- **A Service without a selector** plus your own EndpointSlice points at fixed IP addresses.
- **Why bother?** Move the thing into the cluster later, and only the Service changes.

> ON THE EXAM
>
> 'Give an outside dependency an in-cluster name' is ExternalName, or a selectorless Service.

Headless Services

- **clusterIP: None.** No virtual IP, and kube-proxy does not balance it.
- **DNS returns every ready Pod's IP.** The client chooses which to use.
- **With a StatefulSet,** each Pod also gets its own name: bookings-db-0.bookings-db.
- **Used by** databases and clustered software whose members must address each other individually.

> ON THE EXAM
>
> 'Reach a specific Pod by name' or 'discover all Pod IPs' means headless.

Every Service gets **a DNS name**, so no client ever needs to know an IP address.

CoreDNS

- **The cluster DNS server.** A CNCF graduated project, built from plugins.
- **Runs as a Deployment in kube-system,** behind a Service still named kube-dns.
- **Watches Services and EndpointSlices** through the API server, and answers from them.
- **The kubelet points each Pod** at it, through the Pod's /etc/resolv.conf.

> ON THE EXAM
>
> The Service named kube-dns is served by CoreDNS. The old name stayed for compatibility.

The DNS names Kubernetes creates

RECORD	NAME	RESOLVES TO
Service	booking-api.quayside.svc.cluster.local	The ClusterIP
Headless Service	bookings-db.quayside.svc.cluster.local	Every ready Pod IP
StatefulSet Pod	bookings-db-0.bookings-db.quayside.svc.cluster.local	That one Pod's IP
SRV, named port	_http._tcp.booking-api.quayside.svc.cluster.local	Port number and host
Pod	10-244-1-14.quayside.pod.cluster.local	That Pod's IP

ON THE EXAM

Services read smallest to largest: service, namespace, svc, cluster domain.

Inside a Pod: how short names work

```
$ kubectl exec -n quayside web-7d9f6c-x2k4p -- cat /etc/resolv.conf
nameserver 10.96.0.10
search quayside.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

2 The kube-dns Service's ClusterIP.

3 Search domains: why a bare name like booking-api works.

4 Names with fewer than five dots try the search list first.

THE TAKEAWAY

booking-api resolves in the Pod's own namespace; booking-api.payments reaches another.

dnsPolicy

VALUE	THE POD USES
`ClusterFirst`	Cluster DNS, forwarding outside names upstream. The default.
`Default`	The node's own resolver. Despite its name, not the default.
`ClusterFirstWithHostNet`	Cluster DNS for a Pod on the host network
`None`	Only what you put in dnsConfig

> ON THE EXAM
>
> The value called Default is not the default. ClusterFirst is.

The older way: environment variables

- **The kubelet injects variables** such as BOOKING_API_SERVICE_HOST and _SERVICE_PORT.
- **Only for Services that existed** before the Pod started. Later Services are invisible.
- **DNS has no such ordering problem,** which is why it is the recommended method.

> ON THE EXAM
>
> Two discovery methods: environment variables and DNS. DNS is preferred.

Eight Service decisions. For each, ask: **who needs to reach it, and how?**

> The payment worker calls the booking API. Nothing outside the cluster should ever reach the booking API directly.

Which Service type should the booking API use?

A NodePort

B LoadBalancer

C ExternalName

D ClusterIP

◆ PAUSE AND DECIDE

Which Service type should the booking API use?

A NodePort

Opens a port on every node to the outside.

B LoadBalancer

Creates a public load balancer.

C ExternalName

An alias to something outside; nothing to proxy.

D ClusterIP

Internal only, stable virtual IP: exactly this.

THE SIGNAL

Internal callers only: ClusterIP, the default.

> Quayside's web front end, running on a public cloud, must be reachable from the internet at one stable address.
>
> The team wants the cloud to manage that address and spread traffic across nodes.

Which Service type fits?

A NodePort

B LoadBalancer

C ClusterIP

D A headless Service

◆ PAUSE AND DECIDE

Which Service type fits?

A NodePort

Clients would need node IPs, which change.

B LoadBalancer

One external address, built by the cloud.

C ClusterIP

Not reachable from the internet.

D A headless Service

No virtual IP and no external reach.

THE SIGNAL

Internet, one stable address, cloud-managed: LoadBalancer.

> Quayside's apps use a legacy payments database hosted outside the cluster at payments-db.legacy.example.com.
>
> The team wants apps to use an in-cluster name now, so moving the database later only changes one object.

What should they create?

A An ExternalName Service pointing at that hostname

B A headless Service selecting the database Pods

C A NodePort Service in front of the database

D A ConfigMap holding the database hostname

◆ PAUSE AND DECIDE

What should they create?

A An ExternalName Service pointing at that hostname

A DNS alias: apps use payments-db, the Service holds the real name.

B A headless Service selecting the database Pods

There are no database Pods in the cluster to select.

C A NodePort Service in front of the database

NodePort brings outside traffic in, not the reverse.

D A ConfigMap holding the database hostname

Works, but every app must read it; not a DNS name.

THE SIGNAL

In-cluster name for an outside hostname: ExternalName.

> The three bookings database replicas must find and connect to each other individually, by stable names.
>
> Load balancing across them would break replication.

Which Service suits the replicas?

A A ClusterIP Service with a selector

B A LoadBalancer Service

C A headless Service, clusterIP: None

D An ExternalName Service

◆ PAUSE AND DECIDE

Which Service suits the replicas?

A. A ClusterIP Service with a selector
One virtual IP that load-balances: exactly what breaks replication.

B. A LoadBalancer Service
External, and still load-balanced.

C. A headless Service, clusterIP: None
DNS gives each replica its own name and IP.

D. An ExternalName Service
Points outside the cluster.

THE SIGNAL

'Individually, by name, no load balancing': headless.

> Quayside runs a test cluster on bare-metal servers in its office. There is no cloud and no load-balancer software installed.
>
> Testers on the office network need quick access to the web front end.

What is the simplest option that works?

A. A LoadBalancer Service

B. A NodePort Service

C. A ClusterIP Service

D. An ExternalName Service

◆ PAUSE AND DECIDE

What is the simplest option that works?

A. A LoadBalancer Service
With no cloud or MetalLB, its external IP stays pending.

B. A NodePort Service
A port on every node, reachable from the office network.

C. A ClusterIP Service
Only reachable inside the cluster.

D. An ExternalName Service
Points outwards, not inwards.

THE SIGNAL

Bare metal, no load balancer, quick access: NodePort.

> The web front end sits behind a LoadBalancer Service. The fraud team needs each request's real client IP address.
>
> Application logs show only the nodes' own IP addresses as the source.

What change preserves the client IP?

A Set sessionAffinity: ClientIP on the Service

B Change the Service type to NodePort

C Make it a headless Service instead

D Set externalTrafficPolicy to Local

◆ PAUSE AND DECIDE

What change preserves the client IP?

A Set sessionAffinity: ClientIP on the Service

Makes a client stick to one Pod; the IP is still replaced.

B Change the Service type to NodePort

NodePort forwards the same way; the IP is still replaced.

C Make it a headless Service instead

Headless gives no external access at all.

D Set externalTrafficPolicy to Local

No extra hop, so the client IP is kept.

THE SIGNAL

Real client IP needed: externalTrafficPolicy: Local.

> The bookings database is a StatefulSet in the quayside namespace, with a headless Service called bookings-db.
>
> A tool in the payments namespace must connect to replica 0 specifically.

Which DNS name should it use?

A bookings-db.quayside.svc.cluster.local

B bookings-db-0.bookings-db.payments.svc.cluster.local

C bookings-db-0.bookings-db.quayside.svc.cluster.local

D bookings-db-0.pod.cluster.local

◆ PAUSE AND DECIDE

Which DNS name should it use?

A bookings-db.quayside.svc.cluster.local

Resolves to every replica, not replica 0 alone.

B bookings-db-0.bookings-db.payments.svc.cluster.local

Right shape, wrong namespace: that is the caller's.

C bookings-db-0.bookings-db.quayside.svc.cluster.local

Pod name, then the headless Service, then namespace.

D bookings-db-0.pod.cluster.local

Not a name Kubernetes creates.

THE SIGNAL

One StatefulSet Pod by name: pod.service.namespace.svc.cluster.local.

> The booking API Service resolves correctly in DNS, but every connection to it times out.
>
> Its EndpointSlice lists no endpoints. The Service selects app=booking-api; the Pods are labelled app=booking.

What is the fix?

A Match the Service selector to the Pod labels

B Restart CoreDNS so the Service name is refreshed

C Change the Service type to NodePort

D Restart kube-proxy on every node

◆ PAUSE AND DECIDE

What is the fix?

A Match the Service selector to the Pod labels

No matching Pods means no endpoints. Fix the selector or labels.

B Restart CoreDNS so the Service name is refreshed

DNS already resolves correctly.

C Change the Service type to NodePort

A new type still has no Pods behind it.

D Restart kube-proxy on every node

kube-proxy has no endpoints to route to.

THE SIGNAL

DNS fine, EndpointSlice empty: a selector and label mismatch.

Module 14 in five lines

- **ClusterIP** internal (default); **NodePort** 30000-32767 on every node; **LoadBalancer** one cloud address.
- **ExternalName** is a DNS alias outwards; **headless** returns Pod IPs, no virtual IP.
- **externalTrafficPolicy: Local** keeps the client IP; Cluster spreads evenly.
- **CoreDNS** behind the kube-dns Service; names read **service.namespace.svc.cluster.local**.
- dnsPolicy **ClusterFirst** is the default; the value named Default is not.

One LoadBalancer per Service gets expensive. Next: **Ingress, and its successor the Gateway API**.